

「桌面應用程式開發」 C#程式設計

電機工程學系 李東霖博士

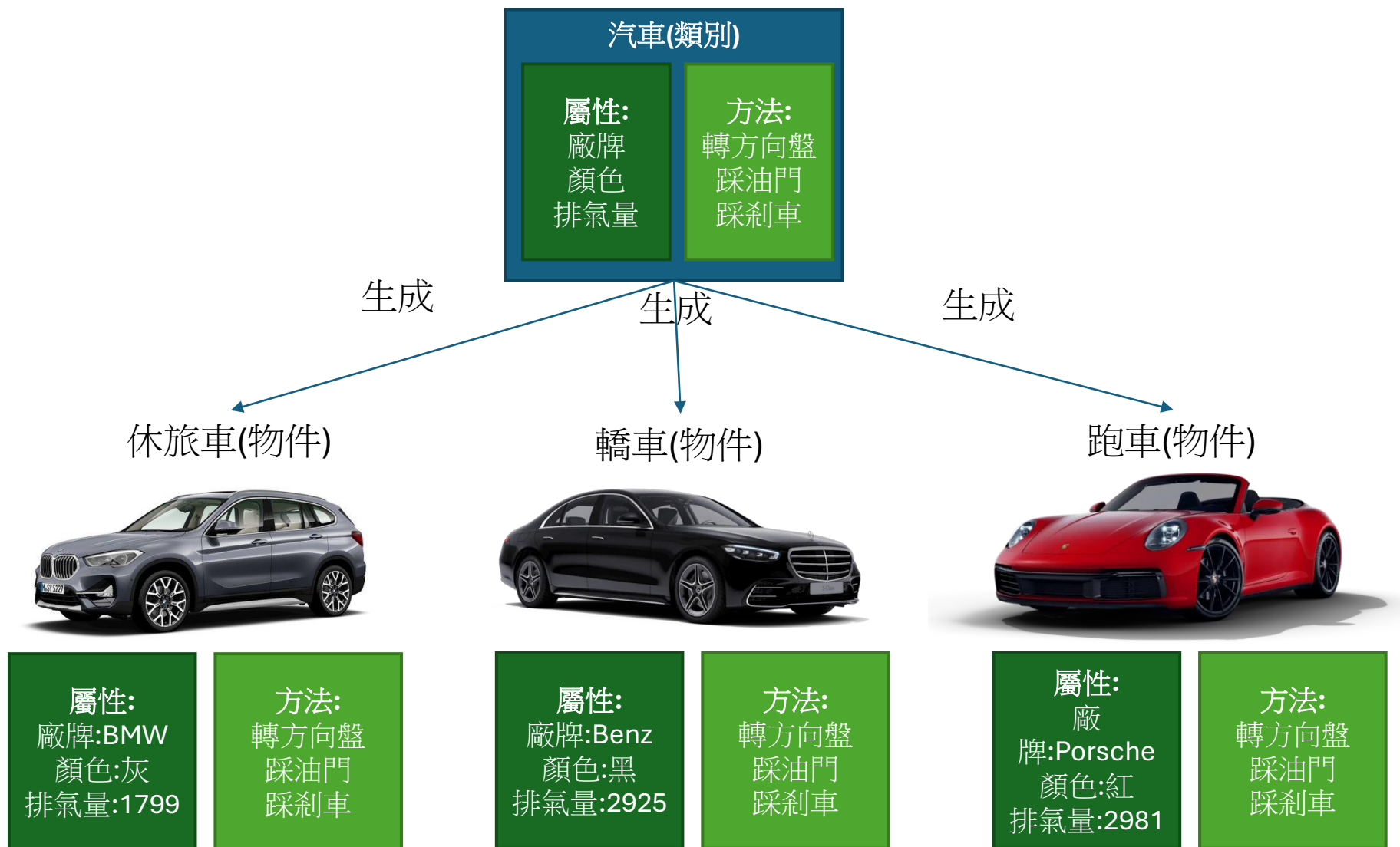
2025 Copyright

C#的基本介紹

C#的基本介紹

- C# (C Sharp) 是一種專為物件導向程式設計的程式設計語言。
- 物件(Object)可以用來代表抽象概念或具體的事物
- 描述物件的3種主要特徵:
 1. 類別(Class)
 - 用來泛指同類性質的不同物件
 - EX. 飛機類別, 汽車類別, 人類別, 狗類別...
 2. 狀態(State)或稱屬性 (Property)
 - 表示物件的特性(attribute), 這些特性代表了一個物件的外觀或某些性質
 - Ex. 顏色, 重量, 形狀, 位置...
 3. 行為(Behavior)或稱方法 (Method)
 - 用來表示物件可產生的動作
 - Ex. 開始, 停止, 移動, 更新...

類別與物件關係圖



物件導向的重要特性

1. 抽象性(**Abstraction**)：可以用來表達抽象概念。
2. 封裝性(**Encapsulation**)：將物件區分為可被外界使用的特性及受保護的內部特性
 - **public**：公用封裝等級。開放給任何程式碼取用。
 - **private**：私用封裝等級。只允許相同類別內的程式碼。
 - **protected**：保護封裝等級。只允許相同類別及衍生類別(類別的類別)的程式碼取用。
3. 繼承性(**Inheritance**)：為了達成重覆使用目的所採取的一種策略。
 - 繼承可讓您建立的類別被重複使用、擴充和修改類別中所定義的方法。
 - 子類別可透過繼承來使用父類別的屬性與方法。
4. 多型性(**Polymorphism**)：是一個非常抽象的名詞，代表著一種彈性。
 - **Ex.** 煮開水，但有多種物品(瓦斯爐、電磁爐、熱水瓶)都可以用不同的方法煮開水。

命名空間(Namespace)

- 它表示著一個識別碼(identifier)的可見範圍。
- 一個識別碼可在多個命名空間中定義，它在不同命名空間中的含義是互不相干的。所以在一個新的命名空間中可定義任何識別碼，它們不會與任何已有的識別碼發生衝突，因為已有的定義都處於其他命名空間中。
- 識別碼在同一命名空間是唯一的。
- Ex.
 - 海大電機系有一位學號10053032的同學叫王小明，文創系也有一位學號1009E001的同學叫王小明。
 - 台大也有一位學號10053032的學生叫王小明

命名空間與類別(Class)的宣告方式

- 以一般而言，C#的物件宣告方式如下：

```
namespace test
{
    public class Obj
    {
        protected int number;
        private void Start()
        {
            ;
        }
    }
    public class subObj:Obj
    {
        int ID;
        void stop()
        {
            ;
        }
    }
}
```

命名空間宣告：namespace [識別名]

類別宣告：封裝等級 class [識別名]

屬性宣告：資料型態 [識別名];

方法宣告：封裝等級 資料型 [識別名](輸入參數)

有繼承的類別宣告：封裝等級 class [識別名]:父類別

沒有註明封裝等級的會預設為public

Tip:

1. 在namespace下的最上層物件封裝等級只能為public

類別的生成與使用

- 使用命名空間:
 - `using test;`
- 物件生成:
 - `Obj t1;`
 - `subObj t2;`
- 物件屬性與方法使用
 - `t1.Start();`
 - `t2.ID`
 - `t2.stop();`
 - `t2.Start();`(繼承父類別的方法)

C/C++/C#的差異

- C語言是一種結構化的程式語言，通常運做在較底層(作業系統、驅動程式、嵌入式系統、解碼引擎)的程式設計上，C語言較接近硬體。
- C++ 語言是附著在C語言之上的一個程式語言，C++是結構化的物件導向程式語言，跟C語言很像，但可以使用物件導向的方式來設計程式，較常用在(驅動程式、嵌入式系統、遊戲引擎)。
- C# 是一種物件導向程式語言，完整的物件導向，近似Java，較常用在桌面程式開發：遊戲設計(Unity, Cocos2D)、Web應用程式與資料庫應用程式。

C, C++通常會用在較接近硬體的程式
C#會用在比較偏應用的程式。

C/C++/C#在Output的差異

- C:

```
printf("Hello World!\n");
```
- C++:

```
cout << "Hello World!\n";  
cout << "Hello World!" << "\n";  
cout << "Hello World!"<< endl;  
cout << 3 + 3;
```
- C#:

```
Console.Write("Hello World!\n");  
Console.WriteLine("Hello World!");  
Console.WriteLine(3 + 3);
```

C/C++/C#在Input的差異

- C:

```
char name[30];  
char myChar;  
scanf("%s%c", &name, &myChar);
```

- C++:

```
int x;  
cin >> x;
```

- C#:

```
string name = Console.ReadLine();  
Int age = Console.Read();  
char in = Console.ReadKey();
```

C/C++/C#在Loop的差異

- C:

```
int myNumbers[5] = {10, 20, 30, 40, 50};  
for (int i = 0; i < 5; i++) {  
    printf("%d\n", myNumbers[i]);  
}
```

- C++:

```
int myNumbers[5] = {10, 20, 30, 40, 50};  
for (int num : myNumbers) {  
    cout << num << "\n";  
}
```

- C#:

```
int[] myNumbers = {10, 20, 30, 40, 50};  
foreach (string i in myNumbers) {  
    Console.WriteLine(i);  
}
```

C/C++/C#在NULL類別的差異

- C: C23以後NULL 被定義為 `void *`。可使用`nullptr`來表示任何類型的空指標

```
nullptr_t* ptr = NULL;
```

- C++: C++11以後NULL 被定義為 `0`。可使用`nullptr`來表示任何類型的空指標

```
int pN = NULL;
```

- C#: 使用關鍵字`null`表示

```
int? x = null;
```

C#程式設計基礎

整數數值類型

C# 類型/關鍵詞	範圍	大小	.NET 類型
sbyte	-128 到 127	帶正負號的8位整數	System.SByte
byte	0 到 255	不帶正負號的8位整數	System.Byte
short	-32,768 到 32,767	有符號的16位整數	System.Int16
ushort	0 到 65,535	不帶正負號的16位整數	System.UInt16
int	-2,147,483,648 至 2,147,483,647	帶正負號的32位整數	System.Int32
uint	0 到 4,294,967,295	不帶正負號的32位整數	System.UInt32
long	-9,223,372,036,854,775,808 至 9,223,372,036,854,775,807	帶正負號的64位整數	System.Int64
ulong	0 到 18,446,744,073,709,551,615	不帶正負號的64位整數	System.UInt64
nint	平臺依賴性（在運行時計算）	帶正負號的32位或64位整數	System.IntPtr
nuint	平臺依賴性（在運行時計算）	不帶正負號的32位或64位整數	System.UIntPtr

浮點數值類型

C# 類型/關鍵詞	近似範圍	精確度	大小	.NET 類型
<code>float</code>	$\pm 1.5 \times 10^{-45}$ 至 $\pm 3.4 \times 10^{38}$	~6-9 位數	4 個位元組	System.Single
<code>double</code>	$\pm 5.0 \times 10^{-324}$ 至 $\pm 1.7 \times 10^{308}$	~15-17 位數	8 個字節	System.Double
<code>decimal</code>	$\pm 1.0 \times 10^{-28}$ 至 $\pm 7.9228 \times 10^{28}$	28-29 位數	16 個字節	System.Decimal

- 常數的類型取決於其後綴：
 - 不含後置詞具有d或D後綴的數值的類型為 `double`
 - 具有 F或f後綴的數值的類型為 `float`
 - 具有 M或的m後綴的數值的類型為 `decimal`

非數值類型

類型	範圍	大小	.NET 類型
char	U+0000 至 U+FFFF	16 位	System.Char

- **char**可以使用下列方式來指定值：
 - 字元常值。
 - **Unicode** 逸出序列，`\u` 後面接著字元碼的四個符號十六進位表示法。
 - 十六進位逸出序列，`\x` 後面接著字元代碼的十六進位表示法。
- 可以用**string**來儲存字串物件

Array/List/Dictionary

- Array(陣列): 是相同型別的集合透過索引去取得元素長度是固定的
- List(串列): 是相同型別的集合透過索引去取得元素長度是任意的
- Dictionary(字典): 是鍵值的集合透過鍵去取得值長度是任意的

Array	長度 5				
index	0	1	2	3	4
	p1	p2	null	null	null
List	長度 2				
index	0	1			
	p1	p2			
Dictionary	長度 2				
key	"Alice"	""Kelly			
	p1	p2			

List串列

- **List** 是一種動態陣列，可用於存儲多個元素。
- 允許您不事先指定大小的情況下添加和刪除值，並提供索引來訪問值。
- 要使用 **List**，您需要引入 **System.Collections.Generic** 命名空間。
- 使用 **List<T>**類別，其中 T 表示要存儲的類型，例如int、string.....。

```
List<int> List = new myList<int>(){ 10, 11, 12 };
```

- 透過Add()、Remove()與Clear()來增減內容
- 透過Insert()來插入內容

Dictionary字典

- 使用 `Dictionary<TKey, TValue>` 類別，其中 TKey 表示鍵(Key)的類型，TValue 表示值(Value)的類型。
- 每個鍵(Key)必須是唯一的。
- 使用 Dictionary，需要引入 `System.Collections.Generic` 命名空間。
- 鍵不能為空引用null，若值為引用類型，則可以為空值
- Key和Value可以是任何型別（string，int，custom class 等）

```
Dictionary<int, string> dict2 = new Dictionary<int, string>{  
    [1] = "C#",  
    [2] = "C++"  
};
```

- 透過Add()、Remove()與Clear()來增減內容

算數運算子

- 遞增運算子 ++
- 遞減運算子 --
- 一元加號和減號運算子
- 乘法運算子 *
- 除法運算子 /
- 餘數運算子 % (可用在浮點數)
- 加法運算子 +
- 減法運算子 -
- 三元條件運算子? :
- 複合指派

邏輯運算子

- 邏輯否定運算子 !
- 邏輯 AND 運算子 &
- 邏輯互斥 OR 運算子 ^
- 邏輯 OR 運算子 |
- 條件式邏輯 AND 運算子 &&
- 條件邏輯 OR 運算子 ||
- 可為 Null 的布林值邏輯運算子
- 複合指派

x	y	x & y	Xx y
true	true	true	true
true	false	false	true
true	null	null	true
false	true	false	true
false	false	false	false
false	null	false	null
null	true	null	true
null	false	false	null
null	null	null	null

位元與移位運算子

- 位元補充運算子 ~
- 左移運算子 <<
- 右移運算子 >>
- 不帶正負號的右移運算子 >>>
- 邏輯 AND 運算子 &
- 邏輯互斥 OR 運算子 ^
- 邏輯 OR 運算子 |
- 複合指派

比較運算子

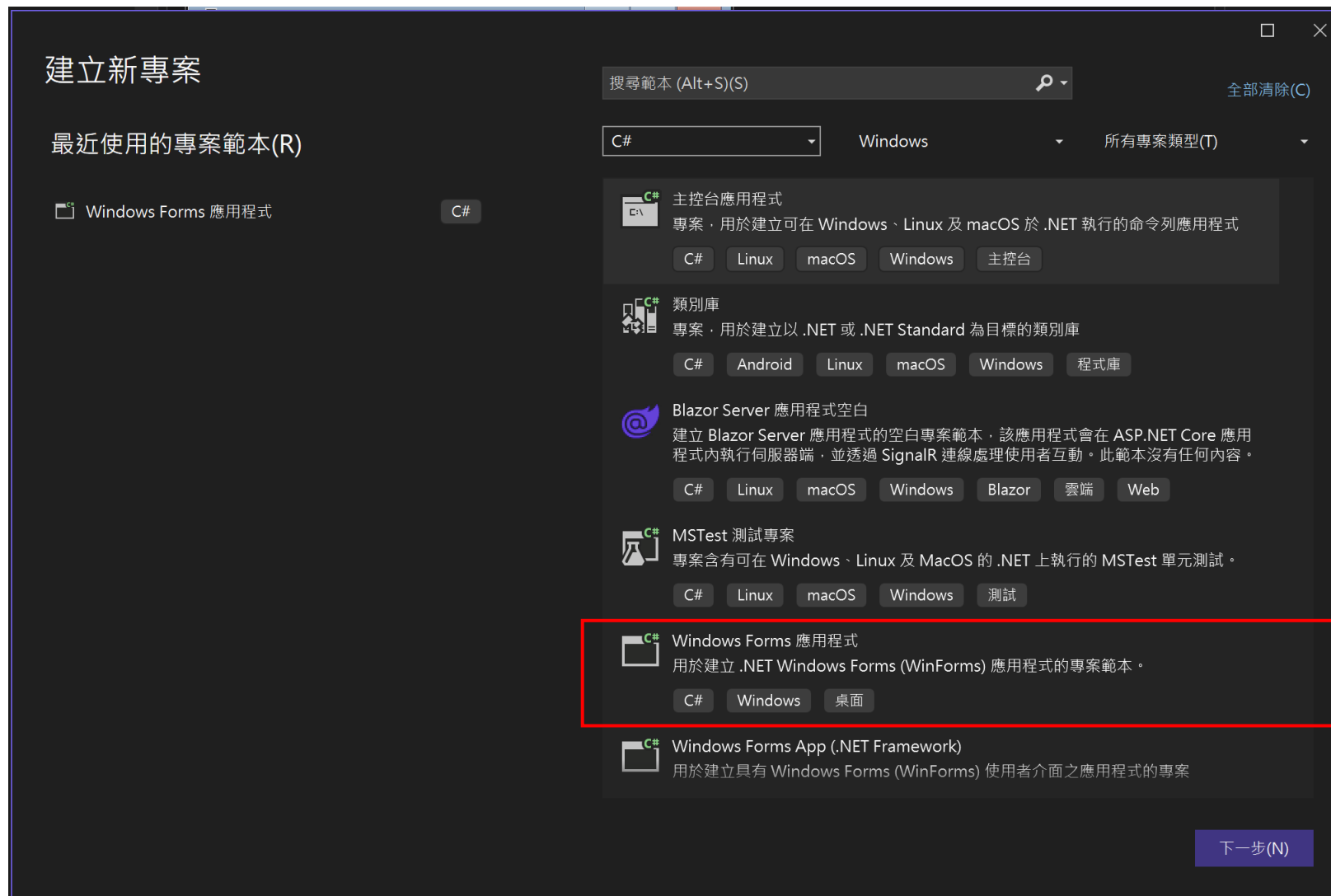
- 等號比較運算子 ==
- 不等比較運算符 !=
- 小於運算子 <
- 大於運算符 >
- 小於或等於運算子 <=
- 大於或等於運算子 >=

空值合併運算子

- ??運算子:
 - `a??b`
 - 當 `a` 為 `null` 時則返回 `b`
 - `a` 不為 `null` 時則返回 `a` 本身
- ??= 運算子:
 - `a ??= b + 2;`
 - 左側運算元評估為 `null` 的時候，才會將右側運算元的值指派給左側運算元。
- ?? 和 ??= 運算子的左方運算元類型不能是不可為 `Null` 的實值型別

Win平台視窗程式開發基礎

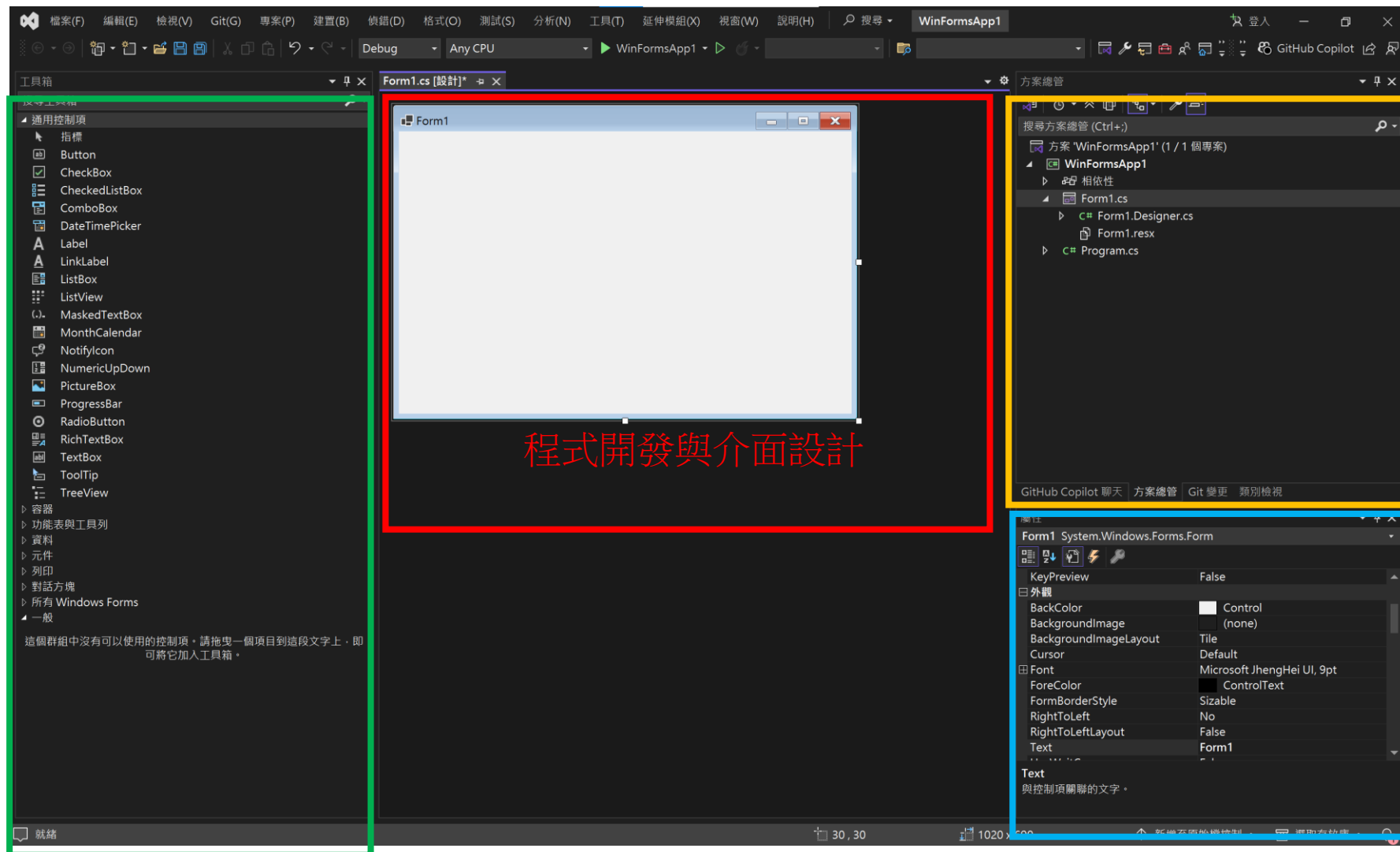
建立專案 – C#的windows Form



開發環境介面介紹

註:未呈現的部份可由選單中的<檢視>下開啟

元件工具箱

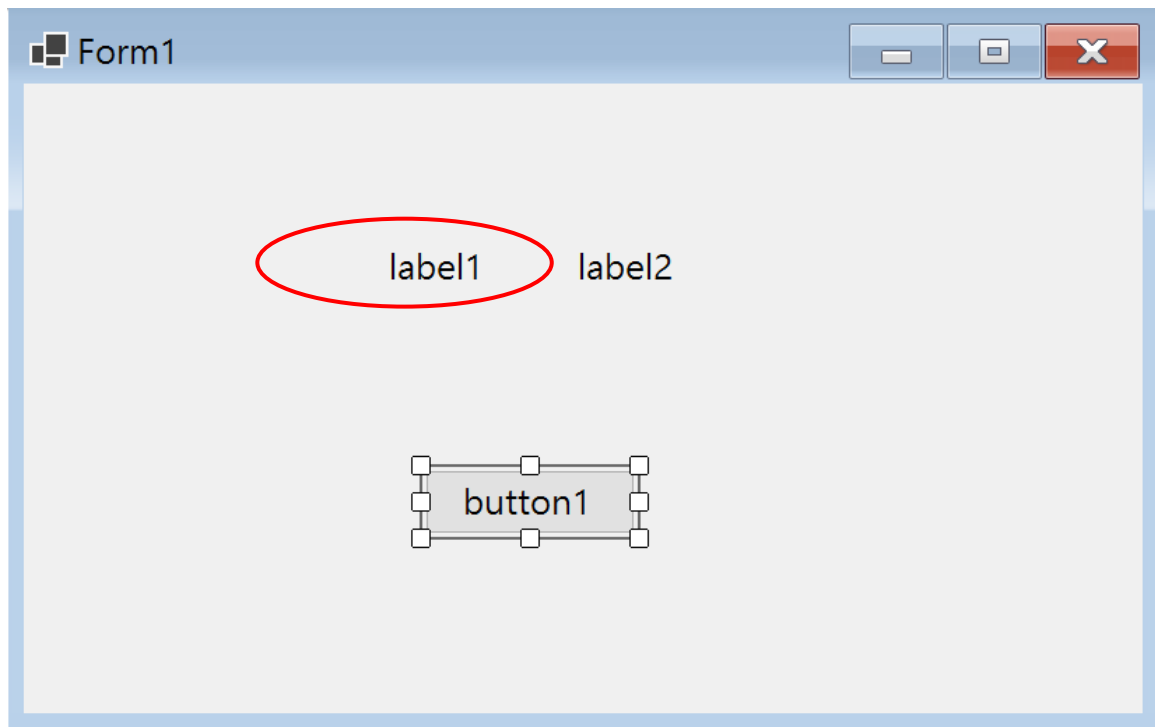


方案總管

元件屬性

Label元件常用屬性

- 常用屬性
 - 外觀
 - BackColor : 背景顏色
 - ForeColor : 文字顏色
 - Font : 字型
 - Text : 顯示文字內容
 - TextAlign : 文字對齊方式
 - 行為
 - Enabled : 是否可使用
 - Visible : 是否顯示
 - 配置
 - AutoSize : 自動元件大小調整
 - Size : 元件預設大小



```
label1.Text = "字串";
```

Button元件常用屬性與應用

- 常用屬性

- 外觀

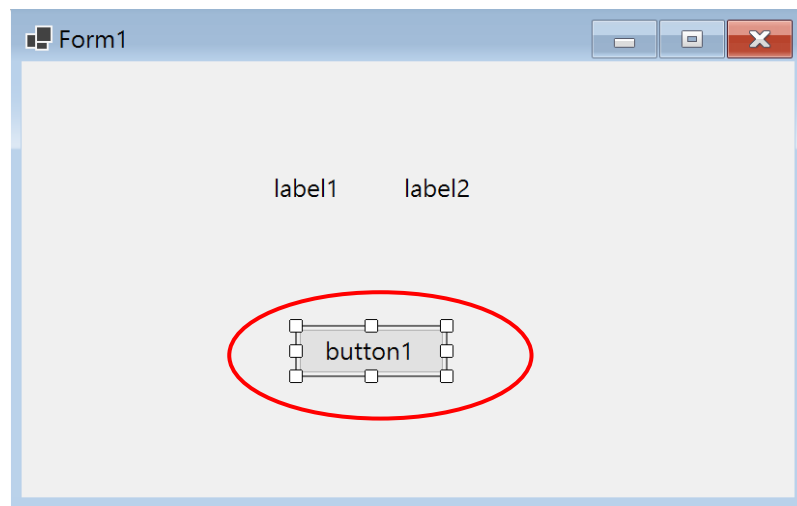
- BackColor : 背景顏色
 - ForeColor : 文字顏色
 - Font : 字型
 - Text : 顯示文字內容
 - TextAlign : 文字對齊方式

- 行為

- Enabled : 是否可使用
 - Visible : 是否顯示

- 配置

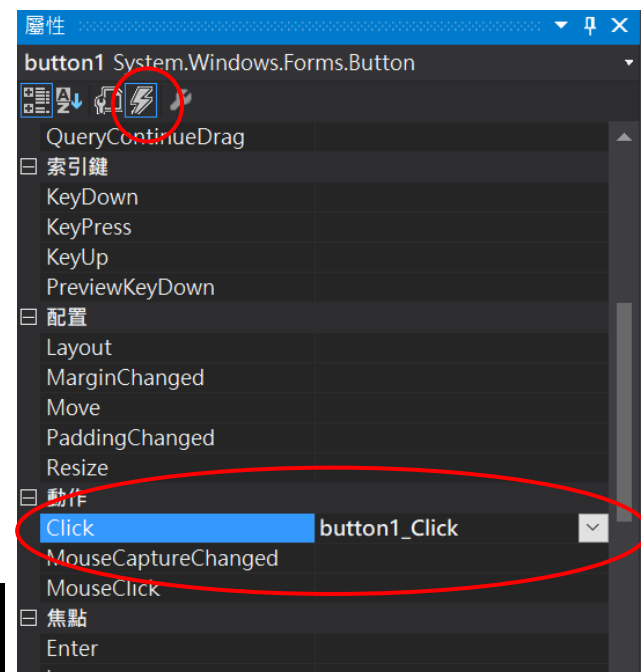
- Size : 元件預設大小



在”屬性視窗”中的”事件”頁面下的
- 動作中

- Click

可設計按鈕後要執行的程式內容
(PS.也可在設計介面中直接點選2次

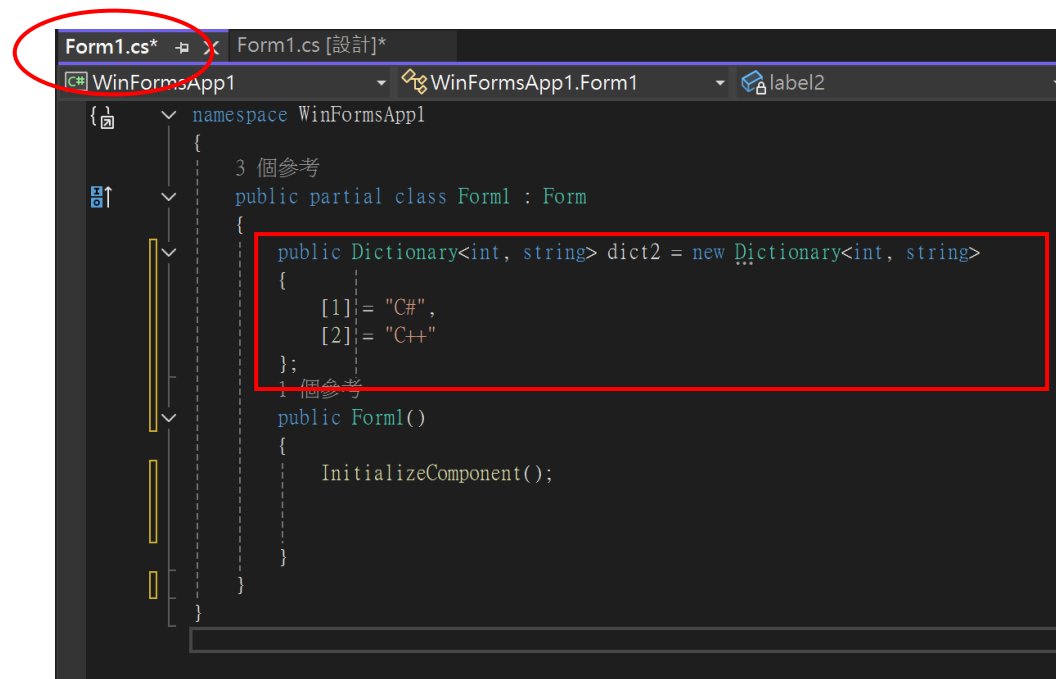


```
private void button1_Click(object sender, EventArgs
{
    MessageBox.Show("HI~~"); //用來開啟訊息方塊
}
```

連動範例(1) – 在視窗建立時創造一個字典

- 點選主視窗的程式碼編輯器
- 在`public partial class Form1 : Form`下添加一個`public`字典

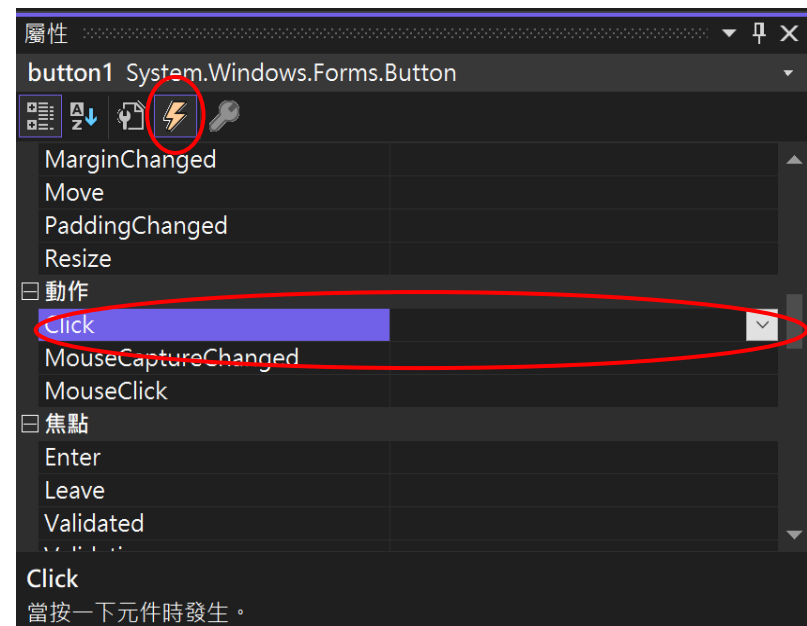
```
public Dictionary<int, string> dict2 = new Dictionary<int, string>
{
    [1] = "C#",
    [2] = "C++"
};
```



連動範例(2) – 設定button動作

- 在button1的元件屬性點選事件下的Click動作兩下
- 在private void button1_Click(object sender, EventArgs e)加如以下程式碼

```
private void button1_Click(object sender, EventArgs e)
{
    label1.Text = dict2[1];
    label2.Text = dict2[2];
}
```



實習時間